# Curriculum Learning for Multi-user Forecasting

Experiment Guideline

Thesis project

25 August 2026

## 1 Research setting

The project tests whether ordering users from easier to harder improves forecasting convergence, final accuracy, or tail-user performance. A run holds the forecasting architecture, optimizer-step budget, batch size, normalization, loss, chronological splits, and evaluation windows fixed; only the user-sampling curriculum changes.

For user $u$, a sampled query date $t$ supplies $X_{u,t} = X_u(t - L, t]$ and $Y_{u,t} = X_u(t, t + H]$. Split ownership is defined by target dates, so validation and test lookbacks may cross the start of their target interval but their targets may not.

## 2 Difficulty and curriculum

The default user difficulty is the median training-window persistence nMSE:

$$d_u = \text{median}_t \frac{\text{MSE}(Y_{u,t}, X_{u,t,L}\mathbf{1}_H)}{(\text{sd}(X_{u,t}) + \varepsilon)^2}.$$

Only accessible training targets enter this score. Constant or non-finite users are removed before ranking by default, and ties use stable source user IDs.

| Method | Contract |
|---|---|
| `uniform` | All users remain equally likely. |
| `easy_to_hard` | Cumulatively reveal users in increasing $d_u$. |
| `hard_to_easy` | Reverse-order control. |
| `random_order` | Seeded random cumulative order. |
| `exposure_matched` | All users are available immediately, with fixed probabilities matching easy-to-hard expected exposure. |

For phase $k$, $S_k$ samples and active set $A_k$, the target exposure is

$$E_u = \sum_k S_k \frac{\mathbf{1}[u \in A_k]}{|A_k|}.$$

The comparison between easy-to-hard and exposure-matched separates temporal ordering from marginal reweighting. The default is five equal-duration phases, an initial 20% of users, and linear pacing; square-root and quadratic pacing are supported.

## 3 Models, metrics, and uncertainty

The standalone forecasting core provides DLinear and PatchTST, global and instance normalization, MSE and normalized losses, exact optimizer-step budgets, and query-date sampling. Primary metrics are global MSE, equal-user MSE, worst-10%-user MSE, convergence curves, difficulty-quantile metrics, and realized versus expected exposure. The reversible instance-normalization type is named `RevIN`, without a redundant "Normalization" suffix.

One run seed is the only randomness identifier for that run. It must fix user ordering, user/window sampling, model initialization, and optimization. Across seed runs, reported dispersion intentionally includes all of those sources.

# 4 Implementation path and provenance

The scientific treatment is isolated in `src/curriculum`: difficulty scores, schedules, controllers, and curriculum-aware fitting. General tensor handling follows the TimeTensors interfaces through separate `data/core.py`, `sampling.py`, `frames.py`, `io.py`, `splits.py`, `statistics.py`, and `loaders.py`. Ordinary construction, fitting, manifests, summaries, and plots remain respectively in `model_loading`, `training`, `pipeline`, `results`, and `visualization`; no sibling checkout is imported.

The source-adapted external packages are pinned as follows:

PatchTST package: `external_models/patchtst`
Upstream: `yuqinie98/PatchTST`
Revision: `204c21efe0b39603ad6e2ca640ef5896646ab1a9`
DLinear package: `external_models/dlinear`
Upstream: `cure-lab/LTSF-Linear`
Revision: `0c113668a3b88c4c4ee586b8c5ec3e539c4de5a6`

The package snapshots are byte-identical to TimeTensors and expose the common `lags, dim, horizon` tensor boundary.

The executed scientific path is:

```
load panel and chronological target splits
compute training-only user difficulty and a stable ordering
construct one fixed-step sampler for the selected curriculum
fit one unchanged model/optimizer across every curriculum phase
evaluate common query dates; aggregate global, user, W10, and quantile metrics
```

# 5 Experiment grid

| Mode | Datasets | Settings | Models | Seeds/budget |
|---|---|---|---|---|
| `test` | Electricity | 504:168 | PatchTST | seed 1, 20 steps |
| `full` | ETTh1, Electricity, Traffic, Solar, Weather, Exchange | 168:24, 336:48, 504:168 | PatchTST | seeds 1–3 |
| `ultra` | full grid | full grid | PatchTST, DLinear | seeds 1–3 |

All five curricula are crossed with each selected configuration. ETTh1 uses all seven non-date variables rather than an OT-only derivative.

# 6 Execution and recovery

The DGX front is `curriculum.slurm`; Selena uses `curriculum_selena.slurm`. Both source the same implementation and default to `STAGES=train,tables`. Submit

```
EXPERIMENT_MODE=test sbatch curriculum.slurm
EXPERIMENT_MODE=full sbatch curriculum.slurm
EXPERIMENT_MODE=ultra sbatch curriculum.slurm
```

Replacing the filename by `curriculum_selena.slurm` preserves the same experiment on Selena. Workflow roots `LOGS_ROOT` and `OUTPUTS_ROOT` default to `logs/` and `outputs/`; the Selena front overrides them to `logs_selena/` and `outputs_selena/`. The front sources `src/slurm/run_curriculum_experiment.sh`. Training and table implementations live under `src/slurm/`. Resubmission allocates or resumes current-manifest runs and executes only incomplete seeds. Use `STAGES=train` or `STAGES=tables` only for recovery.

Slurm workflows never submit a publisher or execute Git commands. After any terminal state, including failure, cancellation, or timeout, the user runs `bash publish_job.sh <job-id>` from the project root. The script verifies `main`, sources `$HOME/codes/proxy.sh` (or `PROXY_SCRIPT_PATH`), and fast-forward pulls `origin/main` before artifact

selection or staging. Lightweight publication selects the exact local or Selena stdout/stderr pair when a job ID is supplied, all log trees otherwise, and only aggregate `outputs*/reports/` artifacts. Detailed publication adds all non-binary outputs and diagnostics. Both tiers exclude `*.pt`, `*.npy`, and `*.cbm`. The script then pushes `origin/main` without opening a pull request. A non-fast-forward pull stops without a merge commit, and unrelated staged paths remain outside the publisher commit.

# 7 Data and artifact contract

The loader discovers `config.json` beside the selected CSV or at an explicit path. Shared fields are top-level, project overrides live under `curriculum_learning`, and explicit run settings override both. For `drop_users`, null inherits, an empty list retains every CSV user, and a nonempty run list replaces the configured default. The project reads CSV directly and does not create or consume TimeTensors dataset caches.

The ordered relative identity below the selected output root is

```
curriculum/dataset/L_H/backbone/method/difficulty/aggregation/
  pacing/phases/initial_fraction/run_n/seed_n/
```

Every named model config owns one directory. Steps, batch size, learning rate, evaluation cadence, splits, strides, and quantile count are pipeline configs; device and Slurm placement are runtime configs. One seed fixes every stochastic choice in a repetition. Mode selects a subset of these paths and is neither a path nor signature field.

Each seed saves resolved and dataset configurations, difficulty and curriculum plans, model state, history, loss plot, global/per-user losses, `results.json`, and `all_losses.pt`. The enclosing `manifest.json` uses `schema_version: 1`, records configs, optional plain provenance, launch attempts, per-seed state, required artifacts, and one of the four overall states `not_run`, `running`, `interrupted`, or `completed`. A seed state may additionally be `ready`. Run identity contains only the schema, identity/model configs, pipeline/experiment configs, and seeds. It never fingerprints or hashes code, Slurm files, data, weights, logs, outputs, or directories; code/data changes are manual rerun decisions. The schema changes only for a deliberate global contract break, and `new` creates another repeat with unchanged parameters.

The default `overwrite_exact` policy skips an exact completion, resumes an exact interruption, and allocates the next `run_n` for a changed pipeline. The overall run remains `running` with `ready_at_utc` while finished seed states are `ready`; completion occurs immediately after that configuration's producer process returns successfully. Later configuration or table failures preserve completed runs and interrupt only unfinished work. The completed manifest is authoritative, without later file hashing or synchronization checks. Tables accept distinct/latest/average pipeline policies and selected/latest/distinct/average repeat policies. Explicit pipeline filters select configurations and must match even for one run; `SELECTED_RUNS.txt` stores only automatic or pinned exact repeats. Reports live under `outputs/reports/curriculum/mode/` and record requested filters and obtained input manifests. Other schema versions are rerun or archived, never read through a compatibility fallback.

# 8 Lightweight validation

Run the schedule, dataset-config, workflow, Torch curriculum, and smoke tests under `src/tests/`. Do not run publication training locally.